Component-Based Design and Analysis of Embedded Systems with UPPAAL PORT

John Håkansson[1], Jan Carlson[2], Aurelien Monot[3], Paul Pettersson[2], and Davor Slutej[2]

[1] Department of Information Technology, Uppsala University, Sweden.
johnh@it.uu.se
[2] Mälardalen Real-Time Research Centre, Västerås, Sweden.**
jan.carlson@mdh.se, paul.pettersson@mdh.se, davor@slutej.com
[3] Ecole des Mines, Nancy, France. aurelien.monot@mines-nancy.org

Abstract. UPPAAL PORT is a new tool for component-based design and analysis of embedded systems. It operates on the hierarchically structured continuous time component modeling language SaveCCM and provides efficient model-checking by using partial-order reduction techniques that exploits the structure and the component behavior of the model. UPPAAL PORT is implemented as an extension of the verification engine in the UPPAAL tool. The tool can be used as back-end in to the Eclipse based SaveCCM integrated development environment, which supports user friendly editing, simulation, and verification of models.

1 Introduction

UPPAAL PORT [4] is a new extension of the UPPAAL tool. It supports simulation and model-checking of the component modelling language SaveCCM [1,6], which has been designed primarily for development of embedded systems in the area of vehicular systems. In SaveCCM, an embedded system is modelled as interconnected components with explicitly defined input and output ports for data and control. A component can be an encapsulation of a system of interconnected components, which externally behaves as a component, or a primitive component. In the latter case the functional and timing behaviour of a component is described as a timed automaton [2].

UPPAAL PORT accepts the hierarchical SaveCCM modelling language, represented in XML format, and provides analysis by model-checking *without* conversion or flattening to the model of network of timed automata normally used in the UPPAAL tool. The hierarchical structure of the model, and the particular "read-execute-write" component semantics adopted in SaveCCM is exploited in the tool to improve the efficiency of the model-checking analysis, which is further improved by a partial order reduction technique [10].

To provide user friendliness, UPPAAL PORT can serve as back-end in the SaveCCM integrated development environment (SAVE-IDE) based on Eclipse, see Fig. 1. We have

** This work was partially supported by the Swedish Foundation for Strategic Research via the strategic research centre PROGRESS.

[4] UPPAAL PORT is available from the web page www.uppaal.org/port.

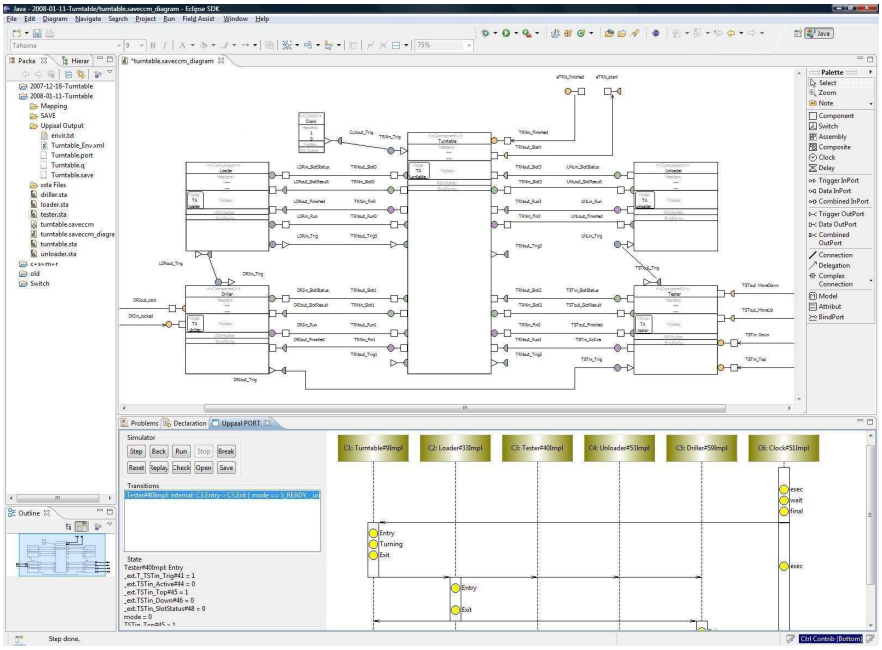

Fig. 1. SAVE-IDE architectural editor (upper view) and UPPAAL PORT simulator (lower view).

developed several plug-ins to integrate the two tools: an editor for timed automata descriptions of the functional and timing behaviour of components, support for mapping internal timed automata variables to external ports, a simulator that can be used to validate the behaviour of a SaveCCM system, and support for verifying reachability and liveness properties formalised in a subset of Timed CTL.

Related work includes for example the BIP component framework [9], where a system is constructed in three layers: behaviour, interaction, and priorities. Partial order techniques for timed automata are described for example in [11,7,5]. See also [10] for additional related work.

2 Real-Time Component Specification

The modelling language employed in UPPAAL PORT is SaveCCM — a component modelling language for embedded systems [1,6]. In SaveCCM, systems are built from interconnected components with well-defined interfaces consisting of input- and output ports. The communication style is based on the pipes-and-filters paradigm, but with an explicit separation of data transfer and control flow. The former is captured by connections between *data ports* where data of a given type can be written and read, and the latter by *trigger ports* that control the activation of components. Fig. 2 shows an example of the graphical SaveCCM notation. Triangles and boxes denote trigger ports and data ports, respectively.

A component remains passive until all input trigger ports have been activated, at which point it first reads all its input data ports and then performs the associated computations over this input and an internal state. After this, the component writes to its output data ports, activates the output trigger ports, and returns to the passive state again. This strict "read-execute-write" semantics ensures that once a component is triggered, the execution is functionally independent of any concurrent activity.

Components are composed into more complex structures by connecting output ports to input ports of other components. In addition to this "horizontal" composition, components can be composed hierarchically by placing a collection of interconnected components inside an enclosing component. From the outside, such a composite component is indistinguishable from a primitive component where the behaviour is given by a single model or piece of code.

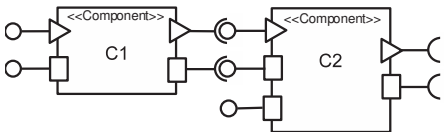

Fig. 2. Composition of two SaveCCM components.

To support analysis and synthesis, a number of quality attributes and models can be associated with a component, such as execution time information, reliability estimates, safety models, etc. For UPPAAL PORT, it is required that each component is associated with a behavioural model consisting of a timed automaton and a mapping between component data ports and automata variables.

3 Model-Checking Real-Time Components

To support the dependability requirements of embedded real-time systems, SaveCCM is designed for predictability in terms of functionality, timeliness, and resource usage. In particular, the independence introduced by the "read-execute-write" semantics can be exploited for analysis purposes using partial order reduction techniques (PORT).

When model-checking, PORTs explore only a subset of the state space. The idea is to define equivalence between traces based on reordering of independent actions, and to explore a representative trace for each equivalence class.This approach has been successful for untimed systems, but for timed automata (TA) the implicit synchronization of global time restricts independence of actions [3,11].

In [10] we have described a PORT for SaveCCM which we have implemented in the UPPAAL PORT tool. As in [3,12] we use local time semantics to increase independence. The structure of a SaveCCM system is used to partition local time-scales, to determine independence of activities, and to construct the Ample-set.

Fig. 3 shows the tool architecture of UPPAAL PORT. The SAVE-IDE integrates an editor for SaveCCM systems in the Eclipse framework, as well as a TA editor to model the timing and behaviour of components. UPPAAL PORT adds support for simulation and verification, using a client-server architecture. When a new SaveCCM system is loaded into the server, the XML parser builds internal representations of UPPAAL TA

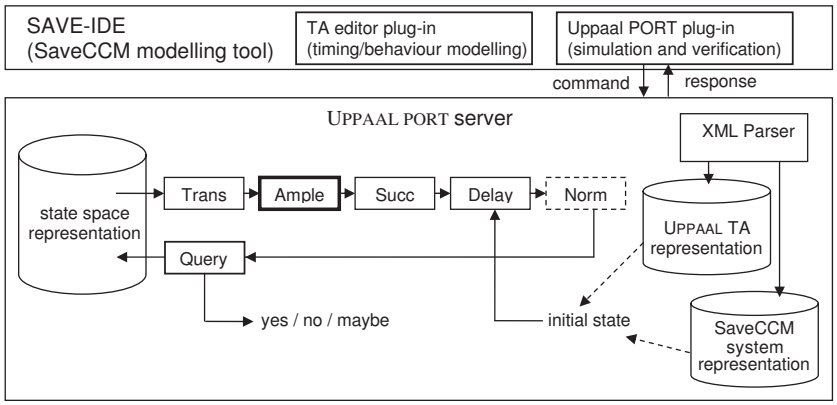

Fig. 3. Overview of the UPPAAL PORT tool architecture.

and the SaveCCM system. By separating the UPPAAL TA representation when a new SaveCCM system is parsed we can reuse much of the source code from the UPPAAL model-checker.

The verification setup is shown in Fig. 3 as pipeline stages connected to the state space representation, as described in [8]. Unexplored states are put into the transition filter (Trans), which computes the enabled transitions. Each transition is forwarded with a copy of the state to the successor filter (Succ), which computes the resulting state after the transition. These two filters of the UPPAAL verifier are extended to implement the SaveCCM semantics. An additional filter (Ample) selects a sufficiently large subset of enabled transitions to be explored in order to model-check the property. This filter implements the PORT described in [10].

The zone representation is replaced with local time zones that are implemented as a data structure similar to Difference Bound Matrices (DBMs), as described for example in [3]. When a component writes data to other components, the local time-scales of participating components are synchronized by the successor filter. In combination with a modified filter (Delay) this implements local time semantics. The purpose of the normalisation filter (Norm) is to ensure that the state space is finite. This remains to be updated in order to handle the 'difference constraints' introduced by using local time.

The transition, successor, and delay filters are used also during simulation to compute possible transitions from the current state of the simulator, and to compute a new state for the simulation when the user selects to make a transition.

4 Case Studies

UPPAAL PORT has so far been applied to some benchmark examples, and two larger case studies. In [1], we present how an early version of UPPAAL PORT is applied to analyse a SaveCCM model of an adaptive cruise controller. A small benchmark of the

partial order reduction technique implemented in the tool is described in [10], showing significant improvement over the standard global time semantics of, e.g., UPPAAL.

We are currently modelling and analysing a turntable production unit [4]. The system has been modelled and the specified requirements (similar to those given in [4]) have been analysed by model-checking.

The turntable system consists of a rotating disc (turntable) with four product slots and four tools in fixed positions around the turntable; the tools operate on the products, as illustrated in Fig. 4. Each slot either holds a single product in some state of the production cycle or is empty. After each 90° rotation of the turntable, the tools are allowed to operate - the turntable is stationary until all tools have finished operating. All slots can hold products and tools are allowed to work in parallel.

The architecture of the system is encapsulated by five SaveCCM components (a turntable and four tools) modelled using SaveCCM timed automata, which are passive and activated by trigger ports. Each component TA wraps C-style code that defines the actual behaviour of the component. This C-style code is directly interpreted by UPPAAL PORT and is suitable as basis for expansion into a production system (the code used in the model for verification has no timeout-detection and error-handling).

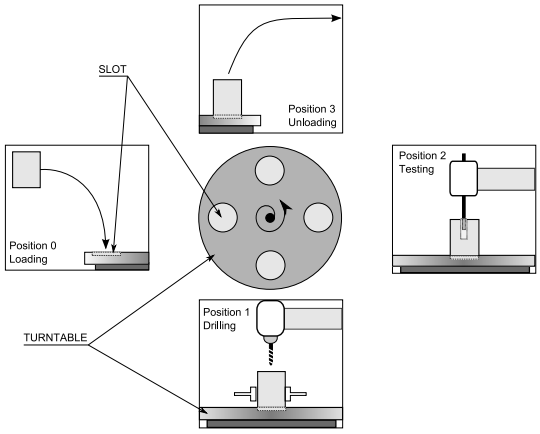

Fig. 4. Turntable system overview.

The control system communicates with the environment by means of external ports that are defined at the root application level. When the code is generated for the target platform these ports are connected to the sensors and actuators. For simulation and verification purposes however, the external ports are mapped to global variables in the environment model. The environment model is constructed using the UPPAAL tool and utilizes UPPAAL timed automata, which, contrasting the SaveCCM TAs, are active.

Properties of safety and liveness are expressed as statements in the UPPAAL requirement specification language. To support more complex queries (involving a sequence of states), a test automaton is constructed in UPPAAL as a part of the environment model. The test automaton is connected to relevant ports in the SaveCCM model, to eliminate the need for test flags and other verification specific (as opposed to functional) additions to the control system model.

Model-checking the properties requires around 16MB at peak and an average of around 3 seconds per verified property (on an Intel T2600 2.16 GHz processor). The verification tool only needs to explore a maximum of 38,166 states to verify properties such as deadlock freedom.

5 Conclusion

In this paper, we have briefly described the new tool UPPAAL PORT that extends the verification engine of UPPAAL with partial order verification techniques for the real-time component language SaveCCM. Our initial experiments with the new verifier have been very encouraging and we are now in progress with evaluating UPPAAL PORT (together with the SaveCCM component modeling language and Save IDE) in a larger case study. As future work, UPPAAL PORT will be expended to support a richer component modeling language with components that may be active, have multiple service interfaces, or use other forms of communication.

References

1. Mikael Åkerholm, Jan Carlson, Johan Fredriksson, Hans Hansson, John Håkansson, Anders Möller, Paul Pettersson, and Massimo Tivoli. The SAVE approach to component-based development of vehicular systems. *Journal of Systems and Software*, 80(5):655–667, May 2007.
2. Rajeev Alur and David L. Dill. A theory of timed automata. *Theoretical Computer Science*, 126(2):183–235, 1994.
3. J. Bengtsson, B. Jonsson, J. Lilius, and W. Yi. Partial order reductions for timed systems. In *Proceedings, Ninth International Conference on Concurrency Theory*, volume 1466 of *Lecture Notes in Computer Science*, pages 485–500. Springer–Verlag, 1998.
4. E. Bortnik, N. Trčka, A.J. Wijs, S.P. Luttik, J.M. van de Mortel-Fronczak, J.C.M. Baeten, W.J. Fokkink, and J.E. Rooda. Analyzing a χ model of a turntable system using Spin, CADP and Uppaal. *Journal of Logic and Algebraic Programming*, 65(2):51–104, 2005.
5. P. Bouyer, S. Haddad, and P.-A. Reynier. Timed unfoldings for networks of timed automata. In S. Graf and W. Zhang, editors, *Proc. of the 4th International Symposium on Automated Technology for Verification and Analysis, ATVA 2006*, number 4218 in Lecture Notes in Computer Science, pages 292–306. Springer–Verlag, 2006.
6. Jan Carlson, John Håkansson, and Paul Pettersson. SaveCCM: An analysable component model for real-time systems. In *Proc. of the 2nd Workshop on Formal Aspects of Components Software (FACS 2005)*, Electronic Notes in Theoretical Computer Science. Elsevier, 2005.
7. F. Cassez, T. Chatain, and C. Jard. Symbolic unfoldings for networks of timed automata. In S. Graf and W. Zhang, editors, *Proc. of the 4th International Symposium on Automated Technology for Verification and Analysis, ATVA 2006*, number 4218 in Lecture Notes in Computer Science, pages 307–321. Springer–Verlag, 2006.
8. Alexandre David, Gerd Behrmann, Kim G. Larsen, and Wang Yi. A tool architecture for the next generation of UPPAAL. In *Proc. of UNU/IIST 10th Anniversary Colloquium*, number 2757 in Lecture Notes in Computer Science. Springer–Verlag, 2003.
9. Gregor Gössler and Joseph Sifakis. Composition for component-based modelling. *Science of Computer Programming*, 55(1-3):161–183, 2005.
10. J. Håkansson and P. Pettersson. Partial order reduction for verification of real-time components. In *Proc. of 1st International Workshop on Formal Modeling and Analysis of Timed Systems*, Lecture Notes in Computer Science. Springer–Verlag, 2007.
11. D. Lugiez, P. Niebert, and S. Zennou. A partial order semantics approach to the clock explosion problem of timed automata. *Theoretical Computer Science*, 345(1):27–59, 2005.
12. Marius Minea. Partial order reduction for model checking of timed automata. In *Proceedings, 10th International Conference on Concurrency Theory*, volume 1664 of *Lecture Notes in Computer Science*, pages 431–446. Springer–Verlag, 1999.